

Optimierungsverfahren

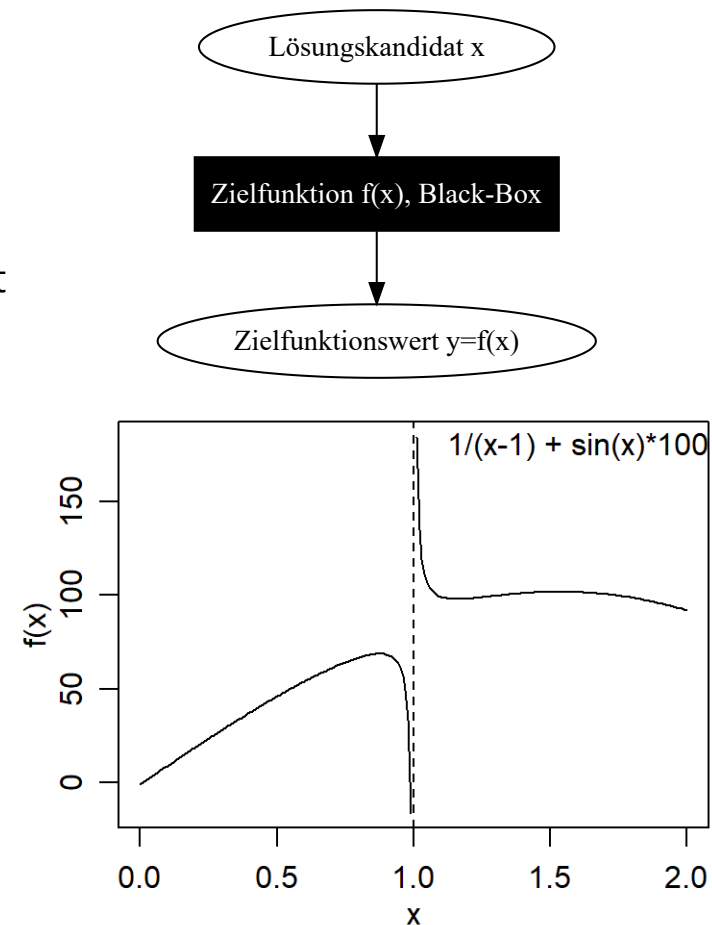
Teil 06 - Metaheuristiken

Martin Zaefferer

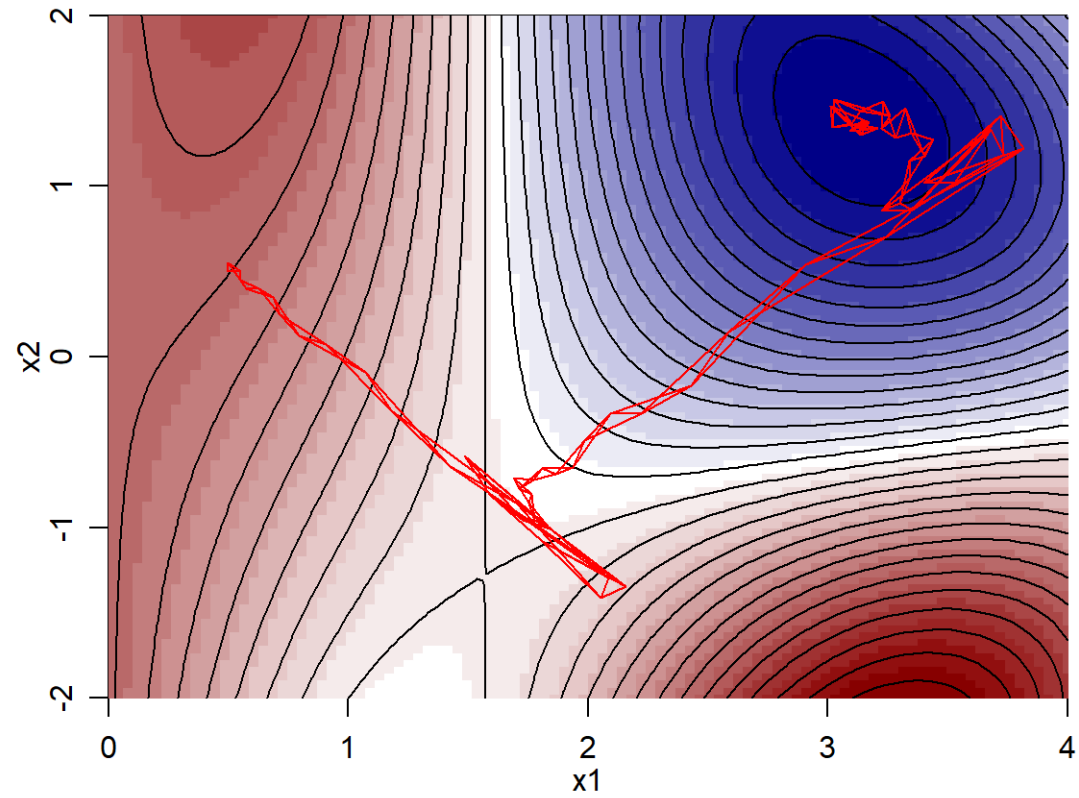
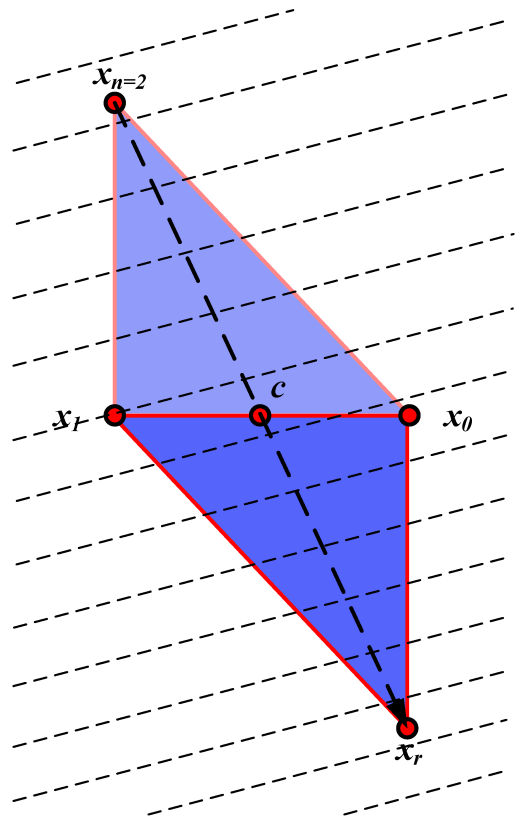
Erinnerung (vorige Vorlesung)

Gradientenfreie Optimierung

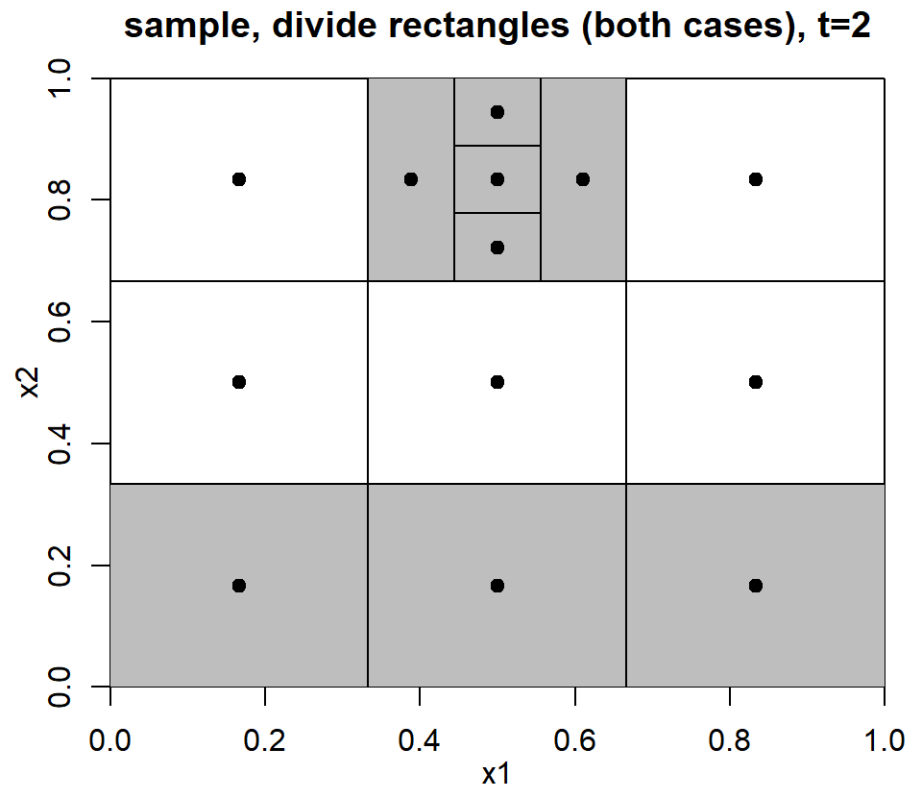
- Bisher: Algorithmen brauchen Gradienten, Hessematrix
- Viele Probleme: nicht differenzierbar
 - Black-box Probleme
 - Innenleben / Details der Zielfunktion unbekannt
 - Wissensgewinn nur über: Auswertung von $f(\vec{x})$
 - Diskontinuierlich Probleme
- Lösungsansatz: Gradientenfreie Optimierung



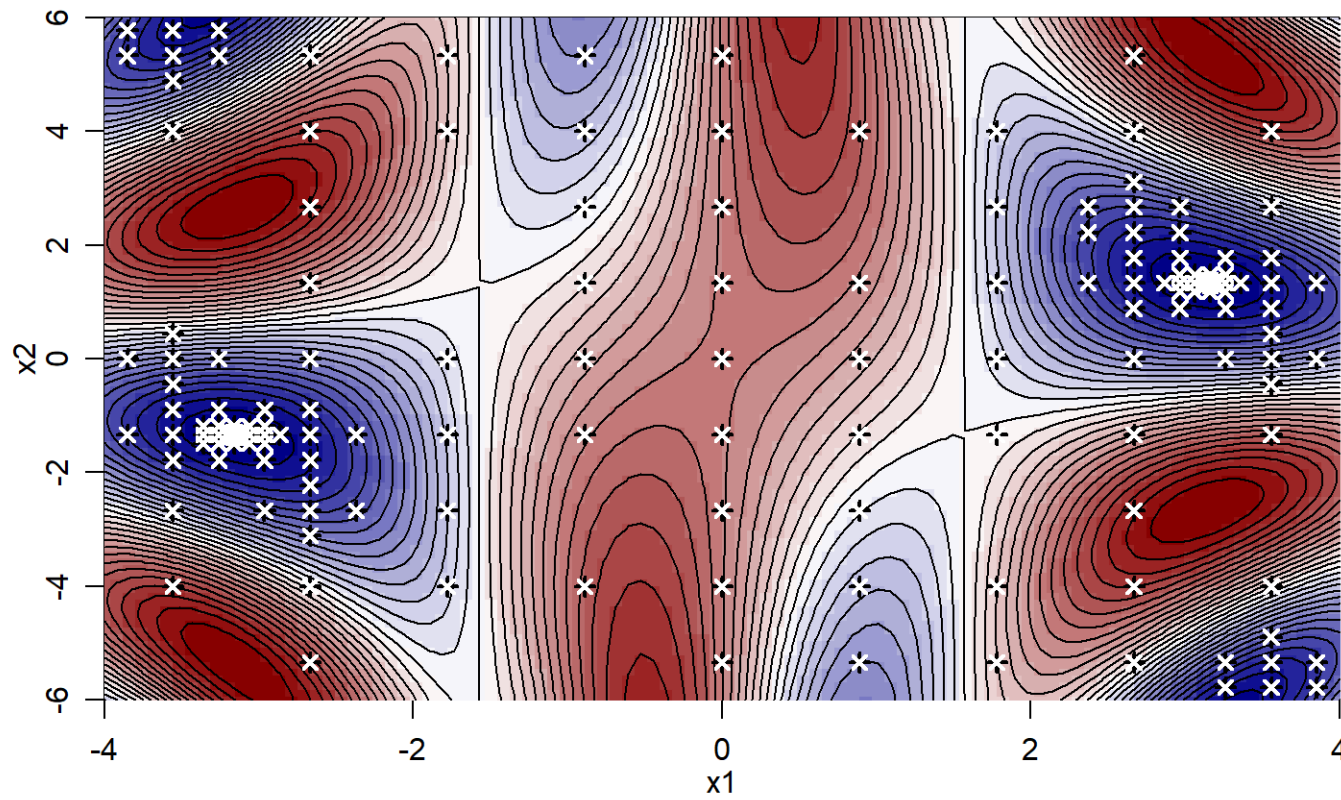
Nelder-Mead Simplex



DIRECT



Direct: Beispiel



Überblick

Eigenschaften von Algorithmen/Methoden

Methode	Global	Gradientenbasiert	Nebenbedingungen	Ziele	Linear	Ansatz	Genauigkeit	Deterministisch	Variablen
Stationäre Punkte / Hesse	✓	✓	ohne	1	✗	analytisch	exakt	✓	$\mathbb{R}^n$
Gradient Descent	✗	✓	ohne	1	✗	numerisch	?	✓	$\mathbb{R}^n$
Simplex	✗	✓	alle	1	✓	numerisch	exakt	✓	$\mathbb{R}^n$ & $\mathbb{Z}^n$
QO	✗	✓	alle	1	Nebenbed.	numerisch	?	✓	$\mathbb{R}^n$
KKT	✓	✓	alle	1		analytisch	exakt	✓	$\mathbb{R}^n$
SQO	✗	✓	alle	1		numerisch	?	✓	$\mathbb{R}^n$
Nelder-Mead	✗	✗	ohne	1	✗	numerisch	approx.	✓	$\mathbb{R}^n$
DIRECT	✓	✗	Grenzen	1	✗	numerisch	approx.	✓	$\mathbb{R}^n$

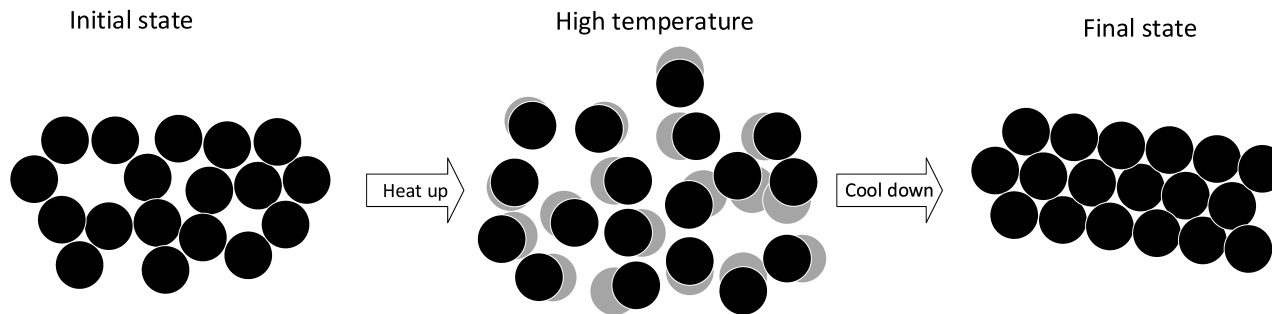
Metaheuristiken und Naturanaloge Algorithmen

Metaheuristiken und Naturanaloge Algorithmen

- Metaheuristik: Gruppe von problemabhängigen, approximativen Solvern
- Viele von der Natur inspiriert (meist Physik und Biologie)
 - Simulated Annealing
 - Particle Swarm Optimization
 - Ant Colony Optimization
 - Evolutionäre Algorithmen

Simulated Annealing

- Inspiriert von physikalischen Prozessen, die zur “Optimierung” von Metallstrukturen verwendet werden
- Hohe Temperatur → viele zufällige Veränderungen in der Struktur
- Langsame Abkühlung → kleinere und kleinere Veränderungen



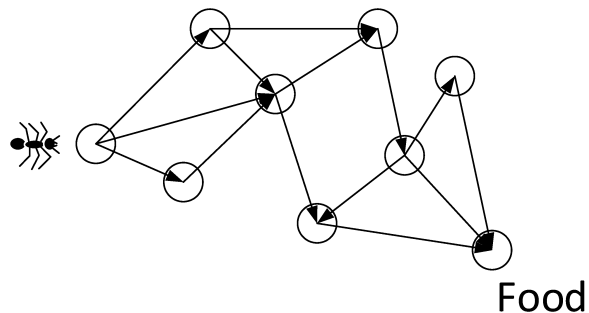
Particle Swarm Optimization / Swarm Intelligence

- Inspiriert durch das Verhalten von Schwärmen (z. B. Vögel, Insekten)
- z.B.: konzertierte Suche nach Nahrungsquellen
- Austausch von Informationen zwischen einzelnen Tieren
- Übersetzung in Algorithmen: jedes Tier repräsentiert einen Lösungskandidat
- Austausch von Informationen: Gute Lösungen beeinflussen, wohin sich schlechtere Lösungen bewegen

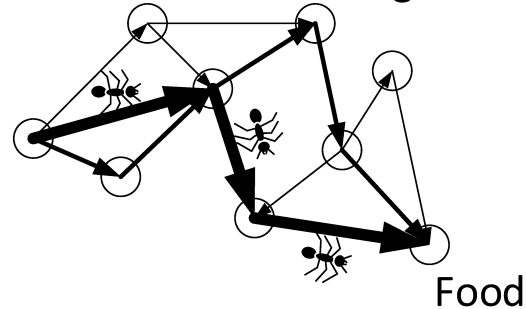
Ant Colony Optimization

- Spezielle Art von Schwarm-Algorithmus
- Ameisen hinterlassen Pheromone entlang ihres Laufpfades
- Ameisen folgen mit größerer Wahrscheinlichkeit stärkeren Pheromonspuren
- Kurze Wege erhalten mehr Pheromone
- Verwendet für z.B. Routing Algorithmen

Possible paths



Promising paths are reinforced due to usage



Evolutionäre Algorithmen

EAs: Frühe Anwendung

- 1960s: Hans-Paul Schwefel - Experimentelle Optimierung einer Zweiphasendüse
- Siehe: <https://ls11-www.cs.tu-dortmund.de/people/schwefel/EADemos/>



- Darstellung: Kombination aus verschiedenen kreisförmigen Scheiben, die in Düsenform angeordnet sind
- Ziel: maximale Umwandlung von Wärmeenergie in kinetische Energie (d. h. maximale Austrittsgeschwindigkeit)

EAs - Analogie

Natur → Algorithmen

- Individuum → Lösungskandidat
- Population → Menge von Lösungskandidaten
- Codiert in DNA → z.B., codiert mit $\vec{x} \in \mathbb{R}^n$
- Mutation → z.B., $\vec{x} + \epsilon$, mit $\epsilon \sim N(0, \sigma)$, und Schrittweite σ
- Rekombination → z.B., $(\vec{x}_{\text{parent1}} + \vec{x}_{\text{parent2}})/2$
- Fitness: Wahrscheinlichkeit des Überlebens / Fortpflanzung → Zielfunktion $f(\vec{x})$

Evolutionäre Algorithmen

Einführendes Material (Grundlagen zu EAs):

- Bäck und Schwefel (1993), An Overview of Evolutionary Algorithms for Parameter Optimization, <https://ls11-www.cs.tu-dortmund.de/people/schwefel/publications/BS93.pdf>
- Whitley (2001), An overview of evolutionary algorithms: practical issues and common pitfalls, <https://www.inf.utfsm.cl/~mcriff/Paper-OR-SW/overview.pdf>
- Bartz-Beielstein et. al (2015), Overview: Evolutionary Algorithms, <https://cos.bibl.th-koeln.de/frontdoor/deliver/index/docId/60/file/bart15jcos.pdf>

EAs: DNA

- Besteht aus Nukleotiden: Cytosin (C), Guanin (G), Adenin (A) oder Thymin (T): z. B. *CGTAGTAC*...
- Mutation: *CGT***AGTAC** → *CGT***CGTAC**
- Rekombination: *CGTAGTAC* + *ACATTGG* → *CGATTGG*

EAs: Binäre Optimierung

- Bitstrings, e.g., 1101010001
- Mutation: 11010**1**0001 → 11010**0**0001 (point-mutation, einzelner bit-flip)
- Rekombination: **111010100001011** + **111100010100101** → **1110101010100101** (single point crossover)

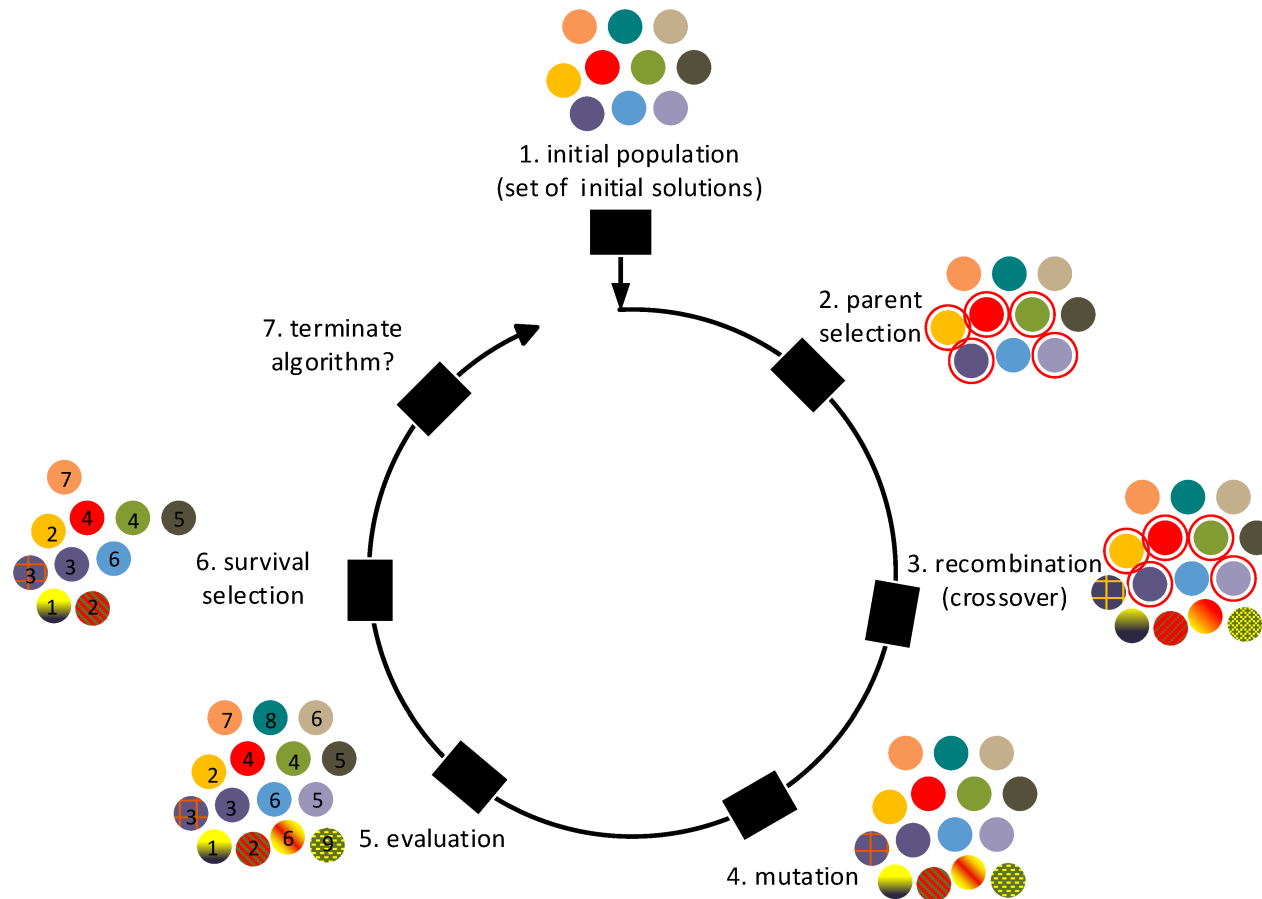
Frage: Fallen Ihnen noch andere Möglichkeiten ein, Bitstrings zu mutieren oder zu rekombinieren??

EAs: Kontinuierlich Optimierung

- Vektoren reeller Zahlen, z.B., $\vec{x} = [0.1, 0.2, 2.6, -1.2]$
- Mutation: $\vec{x} = [0.1, 0.2, 2.6, -1.2] \rightarrow \vec{x} + \epsilon = [0.2, 0.1, 2.5, -1.3]$
- Rekombination:
 $\vec{x}_1 = [0.1, 0.5, 2.6, -1.2]$
 $\vec{x}_2 = [1.3, 1.5, 2.0, 1.2] \rightarrow \vec{x}_3 = (\vec{x}_1 + \vec{x}_2)/2$
 $\vec{x}_3 = [0.7, 1.0, 2.3, 0]$

Frage: Fallen Ihnen noch andere Möglichkeiten ein, reelle Vektoren zu mutieren oder zu rekombinieren?

Evolutionäre Algorithmen



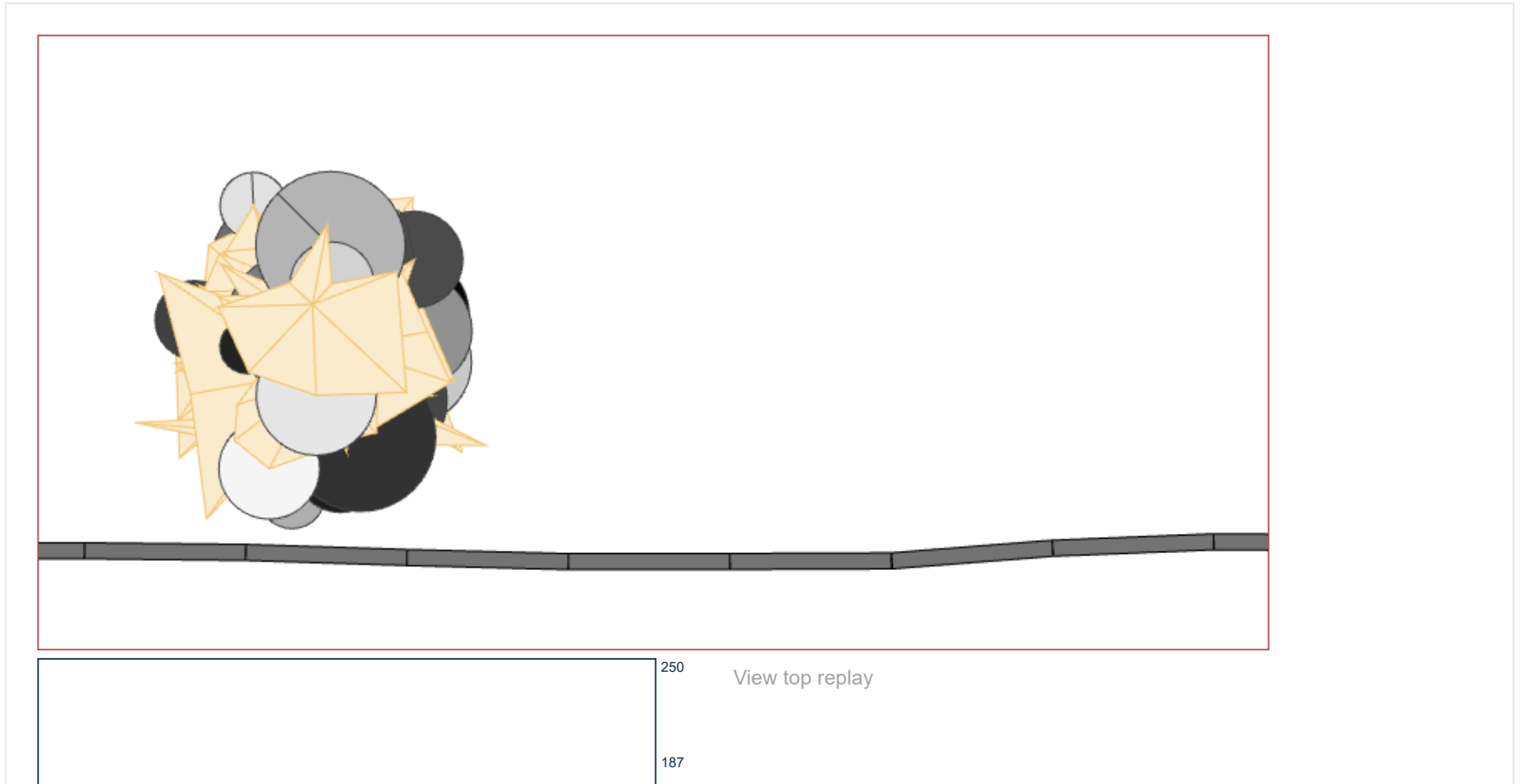
EAs: Demos

Anmerkung: Ich bin nicht für die folgenden Webseiten verantwortlich und habe keinen Einfluss auf Inhalte, Sicherheit oder Datenschutz.

- Demo Videos

- Evolution of a 2D car in Unity – <https://www.youtube.com/watch?v=FKbarpAlBkw>
- Deep Learning Cars – <https://www.youtube.com/watch?v=Aut32pR5PQA>
- Marl/O - Machine Learning for Video Games – <https://www.youtube.com/watch?v=qv6UVOQ0F44>

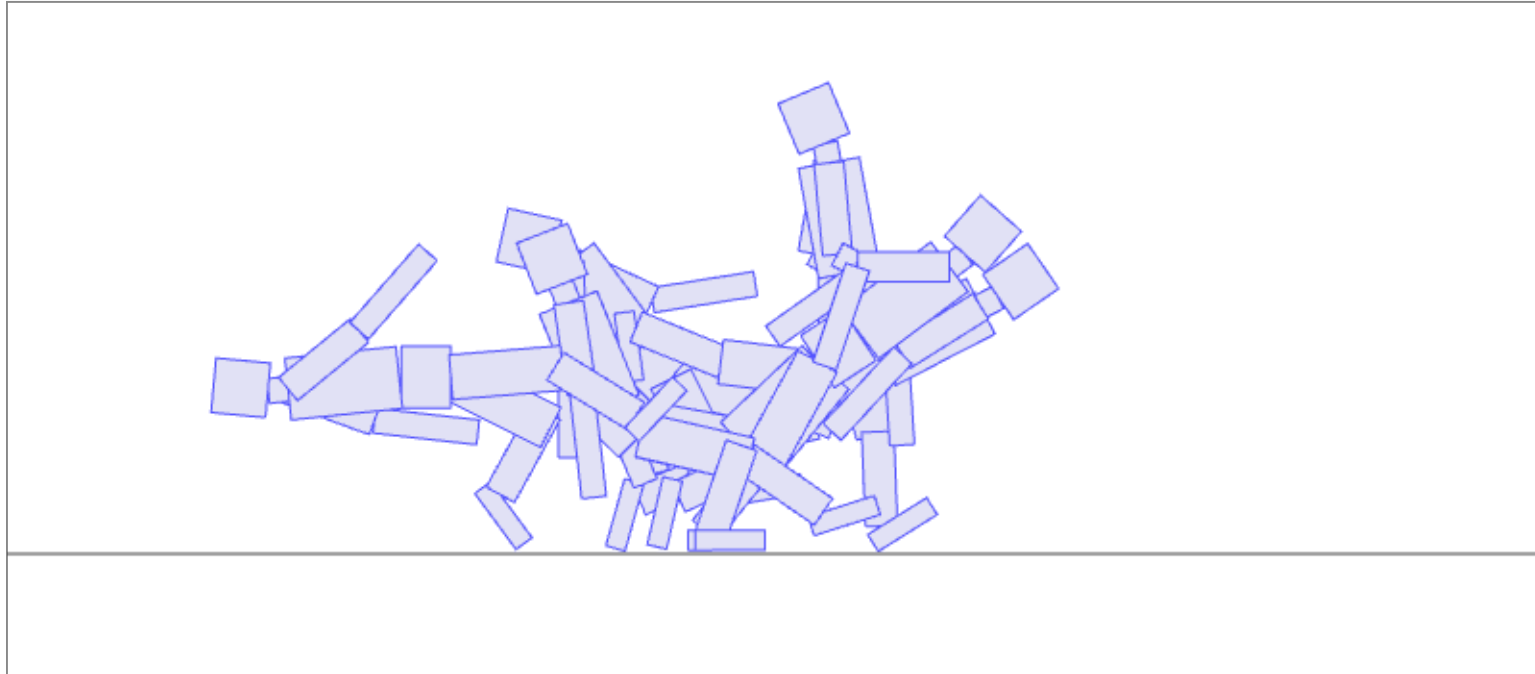
EA Demo: rednught.org - "Genetic Cars 2"



EA Demo: rednught.org - "Genetic Walkers"

Genetic Algorithm Walkers

"Army of QWOP"



Current Generation (0)

Name	Score
Aioefa Kusegi	3.06
Yoquepa Uojoka	1.26
Uuhegi livewa	1.40

Record History

Gen	Name	Score
-----	------	-------

Controls

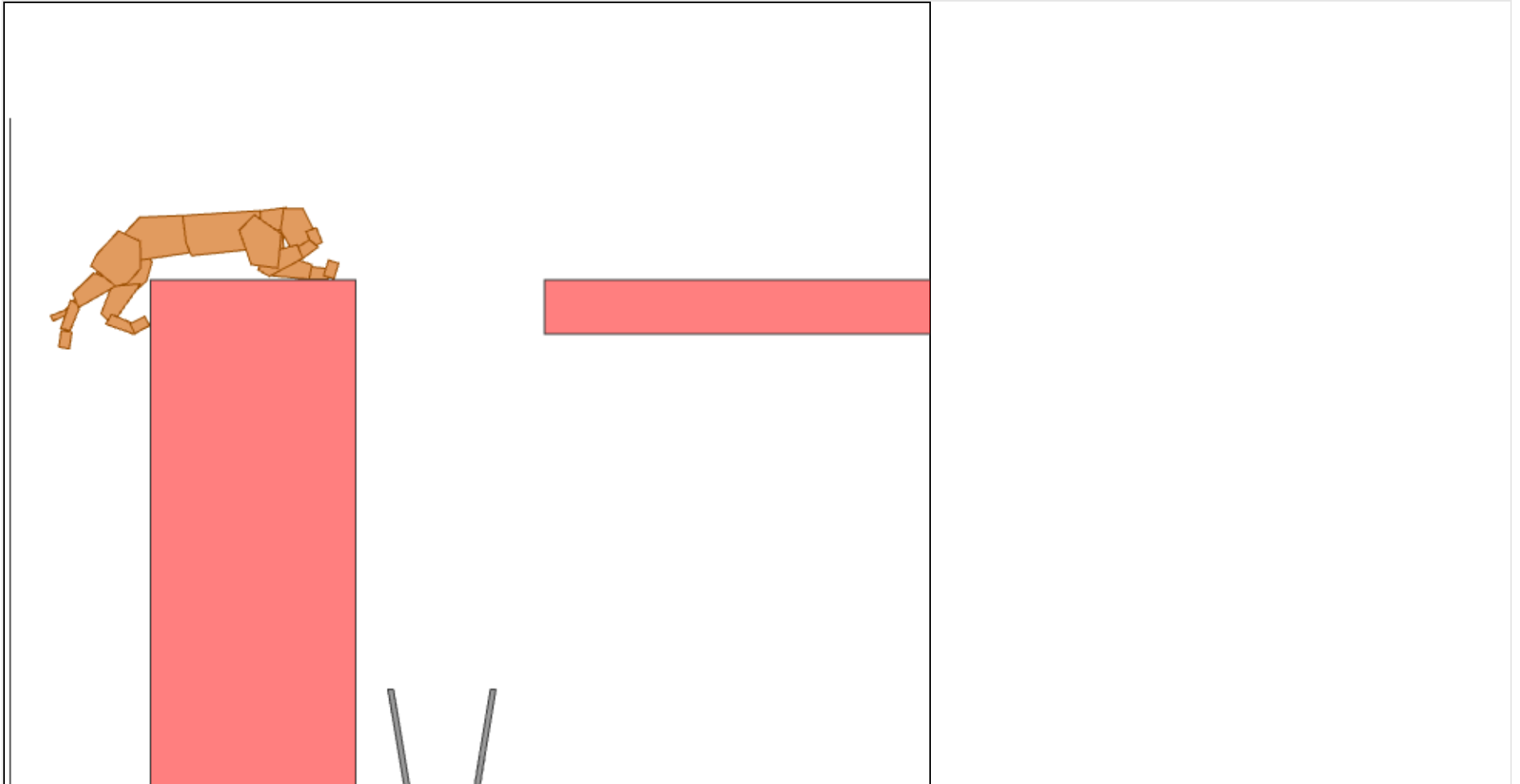
Gene mutation probability 10% ▼
Gene mutation amount 50% ▼
Champions to copy 4 ▼
Motor noise 0.5% ▼

About

What the hell is this?

This observational pastime hopes to evolve walking creatures through genetic algorithms.

EA Demo: rednught.org - “Genetic Cat”



EAs: Anwendungen

- Routenplanung/Navigation
- Terminplanung (Patienten in Krankenhäusern, Maschinen in Fabriken, ...)
- Chemisches Design (Suche nach vielversprechenden chemischen Verbindungen)
- Mechanisches Design (Tragflächen, Automobil)
- Robotik (Steuerungen, Roboterformen / Anordnung)
- Computerspiele (prozedurale Generierung, computergesteuerte Spieler)

EAs: Eigenschaften

- Lokale Suche (aufgrund von kleinen, lokalen Mutationen)
- Globale Suche (aufgrund von mehreren Individuen/Populationen)
- Nicht-linear
- Sowohl mit als auch ohne Nebenbedingungen
- Stochastisch: Selektion, Rekombination und Mutation sind randomisiert
- Keine Gradienten erforderlich
- Kontinuierliche und diskrete Optimierung (abhängig von der Kodierung und den gewählten Operatoren)
- Eine oder mehrere Zielfunktionen sind möglich

Hyperparameter

- EAs haben Hyperparameter (bzw. oft mehr als andere Algorithmen)
- Beispiele:
 - Populationsgröße
 - Mutation: Schrittweite und Wahrscheinlichkeit
 - Anzahl Nachkommen
- Starker Einfluss auf Performanz
- Deshalb:
 - Tuning (Genereller Ansatz, auch für andere Algorithmen)
 - Self-adaptation (spezifischer Ansatz für EAs – Hyperparameter werden Teil des Lösungsvektors)
 - “Parameter control”: Heuristische Regeln zur Einstellung (siehe: 1+1-ES, 1/5-rule)

Probleme von Naturanalogien (EAs und andere)

- Analogien (oder Metapher) aus der Natur als Inspiration für *neue Konzepte*: Gut.
- Analogien als Selbstzweck: Schlecht.
 - Neue Analogie an sich ist aus Sicht der Optimierung kein Neuigkeitswert
 - Durch Änderung der Wortwahl bzw. Formulierung wird verdeckt, dass bestehende Konzepte neu erfunden werden
 - Nicht alle Analogien “funktionieren” bzw. sind sinnvoll zu verallgemeinern (z.B.: auf mehrdimensionale Probleme)
- Leider versteht das nicht jeder, wie folgende Webseite aufzeigt: <https://github.com/fcampelo/EC-Bestiarly>

Überblick

Eigenschaften von Algorithmen/Methoden

Methode	Global	Gradientenbasiert	Nebenbedingungen	Ziele	Linear	Ansatz	Genauigkeit	Deterministisch	Variablen
Stationäre Punkte / Hesse	✓	✓	ohne	1	✗	analytisch	exakt	✓	$\mathbb{R}^n$
Gradient Descent	✗	✓	ohne	1	✗	numerisch	?	✓	$\mathbb{R}^n$
Simplex	✗	✓	alle	1	✓	numerisch	exakt	✓	$\mathbb{R}^n$ & $\mathbb{Z}^n$
QO	✗	✓	alle	1	Nebenbed.	numerisch	?	✓	$\mathbb{R}^n$
KKT	✓	✓	alle	1		analytisch	exakt	✓	$\mathbb{R}^n$
SQO	✗	✓	alle	1		numerisch	?	✓	$\mathbb{R}^n$
Nelder-Mead	✗	✗	ohne	1		numerisch	approx.	✓	$\mathbb{R}^n$
DIRECT	✓	✗	Grenzen	1	✗	numerisch	approx.	✓	$\mathbb{R}^n$
Evolutionäre Algorithmen	✓ (?)	✗ (?)	/	/	✗	numerisch	approx.	✗	/

1+1-Evolution Strategy (1+1-ES)

1+1-ES

- Ein Weg zum Umgang mit Auswirkungen von Algorithmus Parameter: Parameter Control
- Klassisches Beispiel: 1+1-ES und die 1/5-Rule
- Motivation der 1/5-Rule:
 - Wenn erfolgreiche (bessere) Lösungen “zu oft” gefunden werden, ist die Schrittweite zu klein und muss erhöht werden
 - Wenn erfolgreiche (bessere) Lösungen “nicht oft genug” gefunden werden, ist die Schrittweite zu groß und muss verringert werden

1+1-ES und 1/5 rule

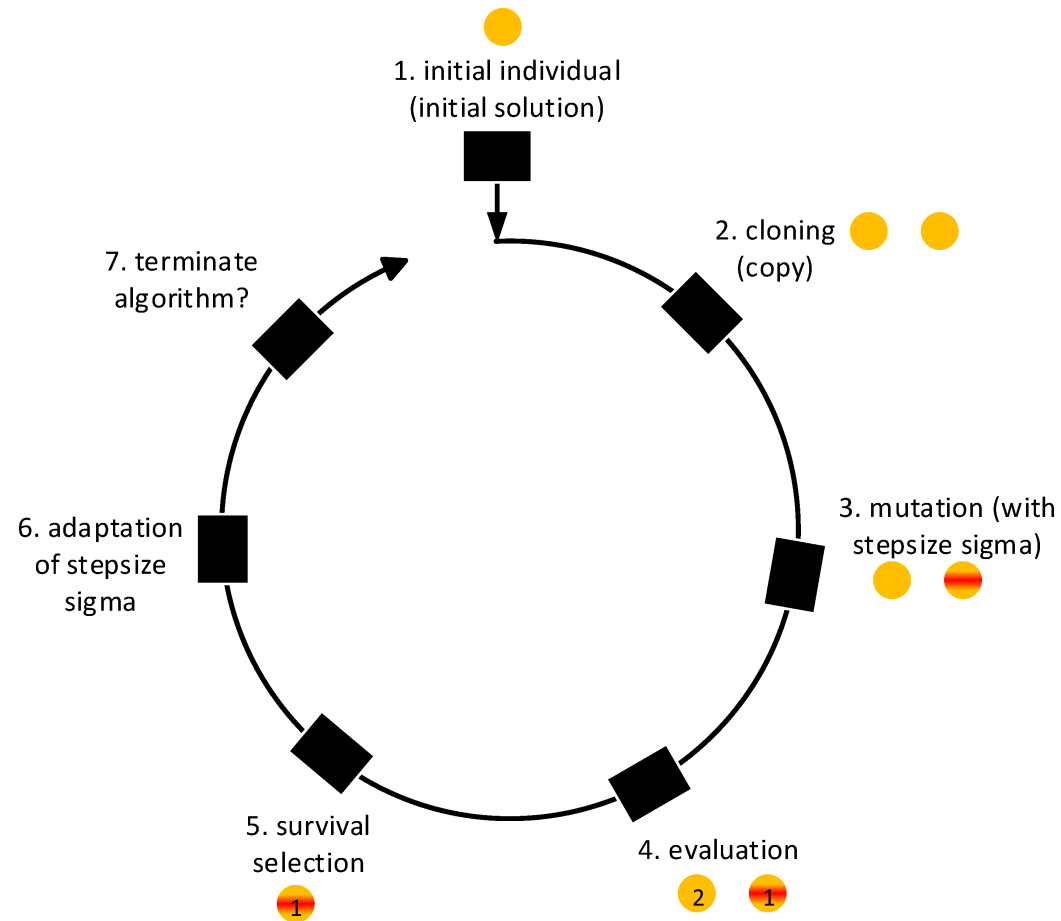
- Theoretisches Ergebnis: Für **bestimmte Probleme** (z.B. die Kugelfunktion $f(\vec{x}) = \sum x_i^2$) bedeutet “zu oft” öfter als 1 Erfolg in 5 Versuchen. (und “nicht oft genug” ist weniger als 1 Erfolg in 5 Versuchen). Die optimale Schrittweite ergibt eine Erfolgsrate von etwa $s = 1/5$.
- Hinweis: Dieses theoretische Ergebnis gilt **nur für die 1+1-ES** (und hinreichend ähnliche Algorithmen) und **nur für bestimmte Probleme**. Nicht allgemeingültig für alle EAs oder alle Optimierungsprobleme.
- Die Regel gilt **nicht** für multimodale (globale) Optimierungsprobleme
- Auch $s = 1/5$ ist nur ein **Näherungswert**, nicht exakt
- Und, es ist theoretisch optimal für $n \rightarrow \infty$

Siehe:

Schumer, Steiglitz (1968), Adaptive Step Size Random Search,
<http://people.duke.edu/~hpgavin/cee201/Schumer-1968.pdf>

Rechenberg (1973), Evolutionsstrategie, Friedrich Frommann Verlag

1+1-ES



1+1-ES: Algorithmus

Parameter: Speicherlänge g , Schrittweitenmultiplikator $a > 1$, initiale Schrittweite $\vec{\sigma} = \vec{1}$

1. Startwert, oder zufällige Startlösung $\vec{x}, y = f(\vec{x})$
2. Clone: Kopieren der aktuellen Lsg $\vec{x}^{\text{next}} = \vec{x}$
3. Mutation: Addieren eines normalverteilten Zufallswertes zu jeder Variable: $x_i^{\text{next}} = x_i^{\text{next}} + \epsilon_i$, mit $\epsilon_i \sim N(0, \sigma_i) = \sigma_i \cdot N(0, 1)$, für alle $i \in 1, \dots, n$
4. Auswertung: $y^{\text{next}} = f(x^{\text{next}})$
5. Selektion: Wenn $y^{\text{next}} < y$: Erfolg (1) speichern in Liste g letzter Schritte G und $\vec{x} = \vec{x}^{\text{next}}, y = y^{\text{next}}$.
Sonst: Fehlschlag (0) speichern. Wenn notwendig, ältesten Speichereintrag löschen.
6. Schrittweitenanpassung: Erfolgsrate $s = \text{Summe(Erfolge)} / g$.
Wenn $s > 1/5$: $\vec{\sigma} = \vec{\sigma} \cdot a$.
Wenn $s < 1/5$: $\vec{\sigma} = \vec{\sigma} \cdot \frac{1}{a}$.
7. Abbruchbedingung? Sonst gehe zu 2.

Überblick

Eigenschaften von Algorithmen/Methoden

Methode	Global	Gradientenbasiert	Nebenbedingungen	Ziele	Linear	Ansatz	Genauigkeit	Deterministisch	Variablen
Stationäre Punkte / Hesse	✓	✓	ohne	1	✗	analytisch	exakt	✓	$\mathbb{R}^n$
Gradient Descent	✗	✓	ohne	1	✗	numerisch	?	✓	$\mathbb{R}^n$
Simplex	✗	✓	alle	1	✓	numerisch	exakt	✓	$\mathbb{R}^n$ & $\mathbb{Z}^n$
QO	✗	✓	alle	1	Nebenbed.	numerisch	?	✓	$\mathbb{R}^n$
KKT	✓	✓	alle	1		analytisch	exakt	✓	$\mathbb{R}^n$
SQO	✗	✓	alle	1		numerisch	?	✓	$\mathbb{R}^n$
Nelder-Mead	✗	✗	ohne	1	✗	numerisch	approx.	✓	$\mathbb{R}^n$
DIRECT	✓	✗	Grenzen	1	✗	numerisch	approx.	✓	$\mathbb{R}^n$
Evolutionäre Algorithmen	✓ (?)	✗ (?)	/	/	✗	numerisch	approx.	✗	/
(1+1)-ES	✗	✗	ohne	1	✗	numerisch	approx.	✗	$\mathbb{R}^n$

R Anwendung

- Siehe AnwendungOV-06-Meta.Rmd

CMA-ES

CMA-ES

Covariance Matrix Adaptation Evolution Strategy (CMA-ES)

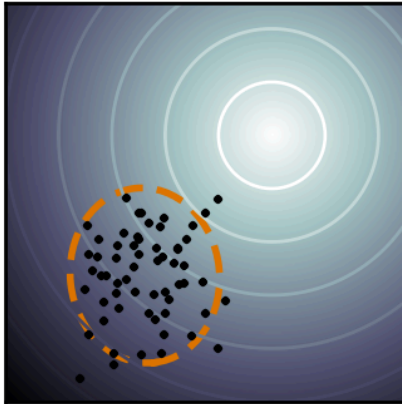
- State-of-the-art Algorithmus für kontinuierliche, evolutionäre Optimierung
- Evolutionsstrategie, die eine fortlaufend veränderte Kovarianzmatrix verwendet, um sich der idealen Verteilung von Individuen im Suchraum anzunähern

Hilfreiches Material:

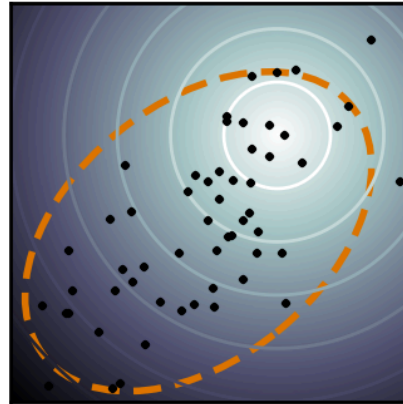
- CMA-ES Webpage <http://cma.gforge.inria.fr/>
- Hansen (2016), The CMA Evolution Strategy: A Tutorial, <https://arxiv.org/pdf/1604.00772.pdf>

CMA-ES: Algorithmus

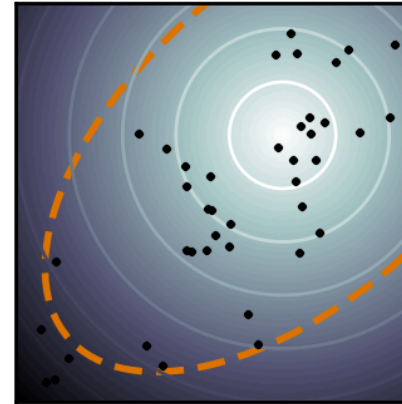
Generation 1



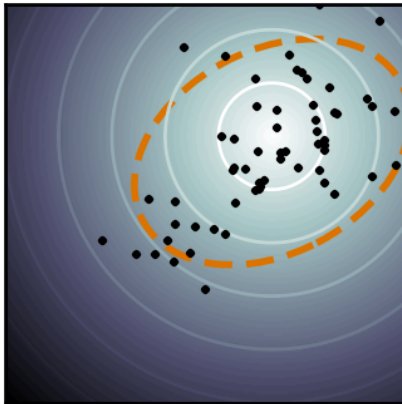
Generation 2



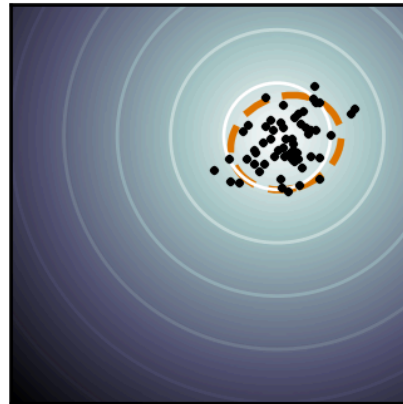
Generation 3



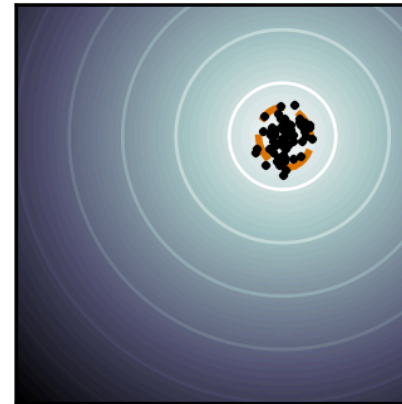
Generation 4



Generation 5



Generation 6



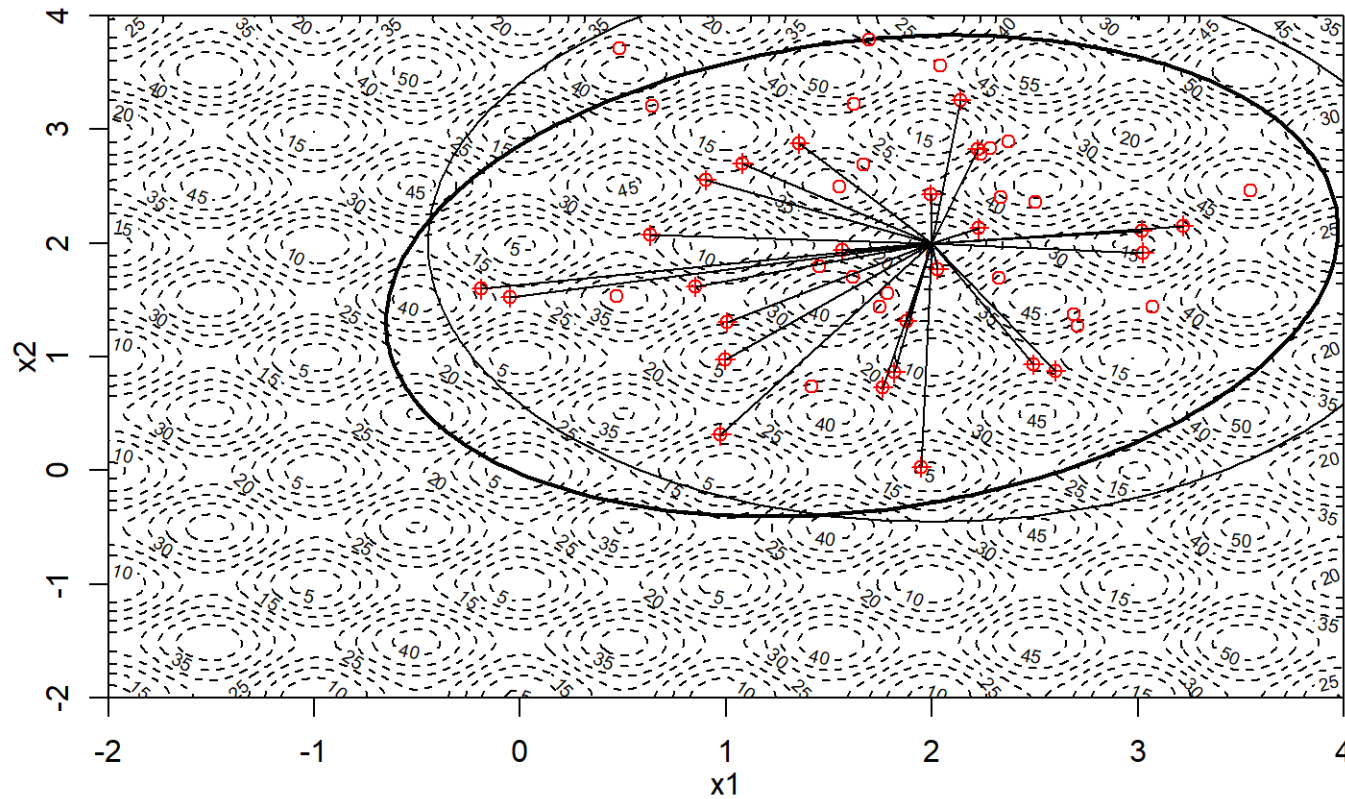
CMA-ES Details: Rastrigin function

- Beispiel zuvor: uni-modal. Multi-modal?
- Ja, geht. Demonstration mit Rastrigin Funktion

$$f(\vec{x}) = 10n + \sum_{i=1}^n x_i^2 - 10 \cos(2\pi x_i)$$

- Aber: Die CMA-ES funktioniert gut wenn eine globale Struktur ausgenutzt werden kann, wie bei der Rastrigin Funktion.
- Probleme mit weniger starker globaler Struktur: Erfordert Neustarts. Oder sehr große Populationen.

CMA-ES Demo: Rastrigin



Überblick

Eigenschaften von Algorithmen/Methoden

Methode	Global	Gradientenbasiert	Nebenbedingungen	Ziele	Linear	Ansatz	Genauigkeit	Deterministisch	Variablen
Stationäre Punkte / Hesse	✓	✓	ohne	1	✗	analytisch	exakt	✓	$\mathbb{R}^n$
Gradient Descent	✗	✓	ohne	1	✗	numerisch	?	✓	$\mathbb{R}^n$
Simplex	✗	✓	alle	1	✓	numerisch	exakt	✓	$\mathbb{R}^n$ & $\mathbb{Z}^n$
QO	✗	✓	alle	1	Nebenbed.	numerisch	?	✓	$\mathbb{R}^n$
KKT	✓	✓	alle	1		analytisch	exakt	✓	$\mathbb{R}^n$
SQO	✗	✓	alle	1		numerisch	?	✓	$\mathbb{R}^n$
Nelder-Mead	✗	✗	ohne	1		numerisch	approx.	✓	$\mathbb{R}^n$
DIRECT	✓	✗	Grenzen	1	✗	numerisch	approx.	✓	$\mathbb{R}^n$
Evolutionäre Algorithmen	✓ (?)	✗ (?)	/	/	✗	numerisch	approx.	✗	/
(1+1)-ES	✗	✗	ohne	1	✗	numerisch	approx.	✗	$\mathbb{R}^n$
CMA-ES	✓ (?)	✗	ohne	1	✗	numerisch	approx.	✗	$\mathbb{R}^n$

Surrogate Model-Based Optimization

Überblick: SMBO

- Surrogate Model-Based Optimization (SMBO)
 - Klasse von Optimierungsalgorithmen für “teure” Zielfunktionen
 - Ziel: Reduziere Kosten der Zielfunktionsauswertung
 - Ersetzen durch weniger “teures” surrogate model (ML Modell)
- Combines approaches von different fields
 - Design of experiment (Versuchsplanung)
 - Statistik / Machine Learning
 - Optimierungsalgorithmen

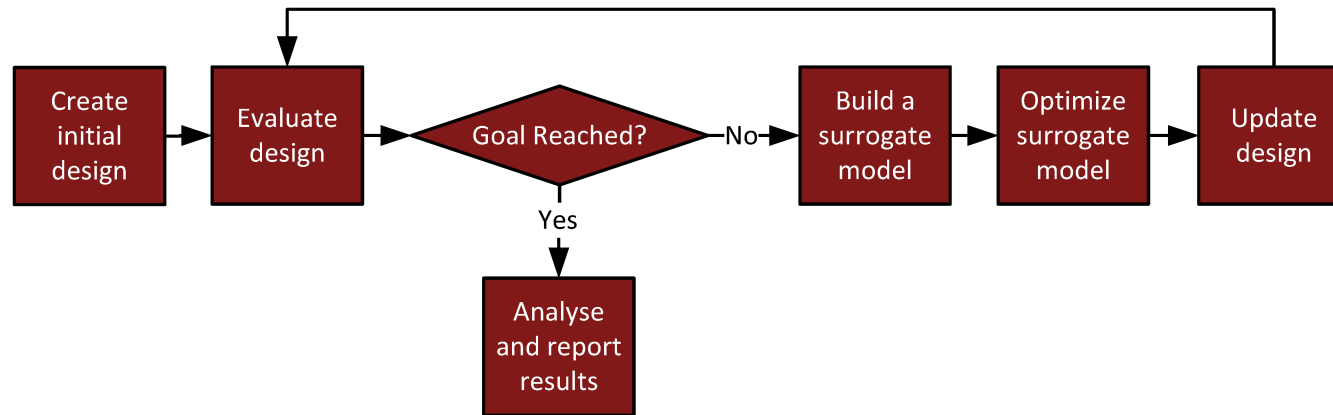
Anwendung

- Zielfunktionen mit Kosten
 - Zeitaufwändig
 - Material / Personalkosten
 - ...
- Anwendungsbeispiele
 - Tuning von Algorithmen (ML)
 - Engineering design (z.B. Automotive, Aerospace)

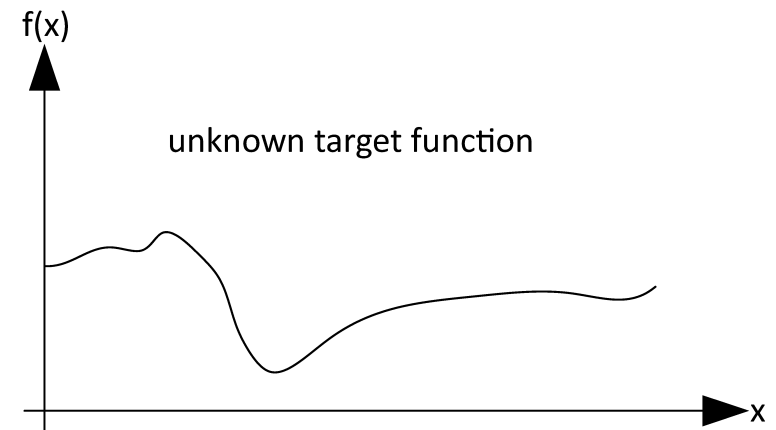
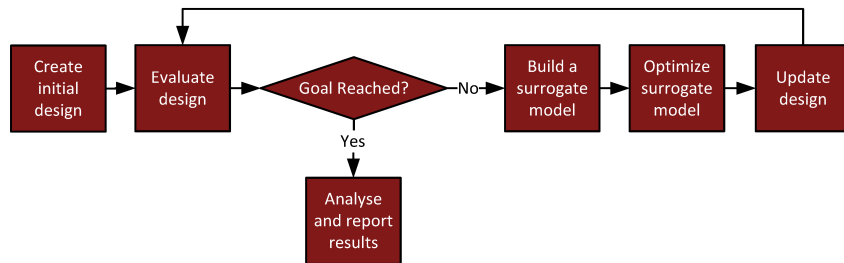
Quellen

- Mockus et al. (1978), The Application of Bayesian Methods for Seeking the Extremum, in Towards Global Optimization, North-Holland Publishing Company,
- Jones et al. (1998), "Efficient Global Optimization of Expensive Black-Box Functions," Journal of Global Optimization 13, no. 4, <https://doi.org/10.1023/A:1008306431147>
- Forrester et al. (2008), Engineering Design via Surrogate Modelling, Wiley, <https://doi.org/10.1002/9780470770801>
- Bartz & Zaefferer (2017), Model-based methods for continuous and discrete global optimization, Applied Soft Computing 55, <https://doi.org/10.1016/j.asoc.2017.01.039>

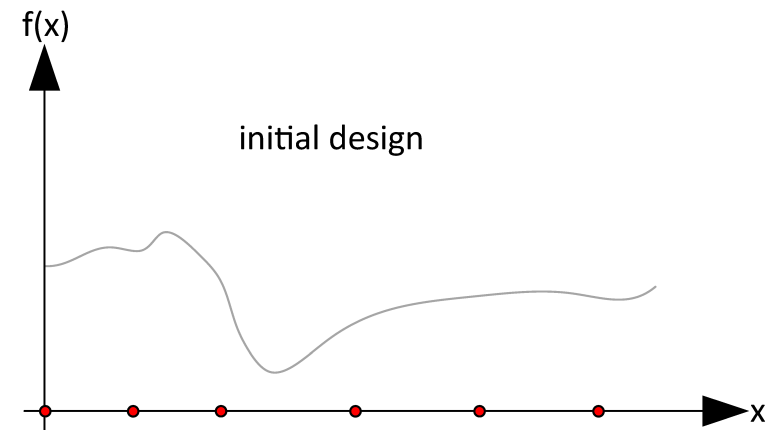
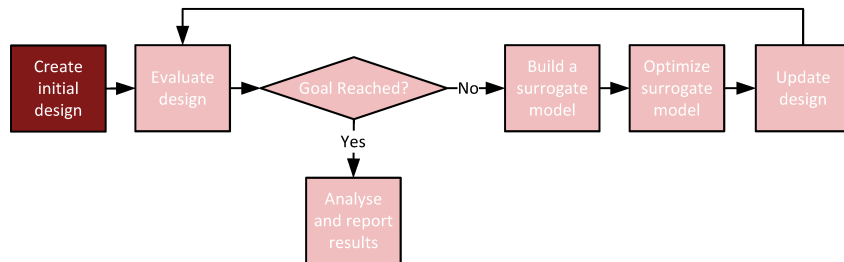
SMBO Prozess



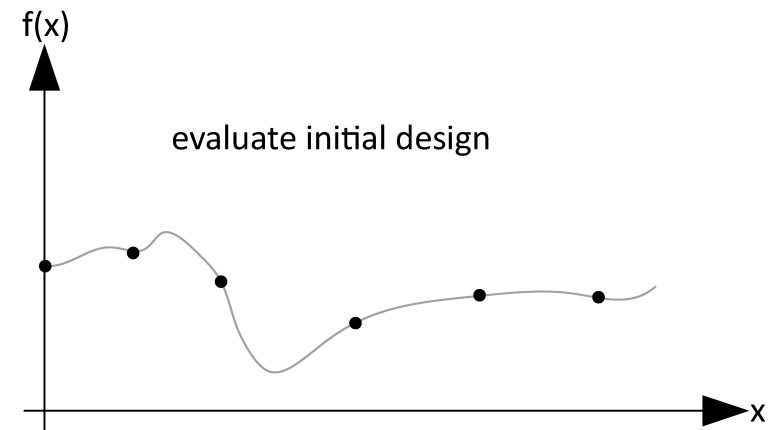
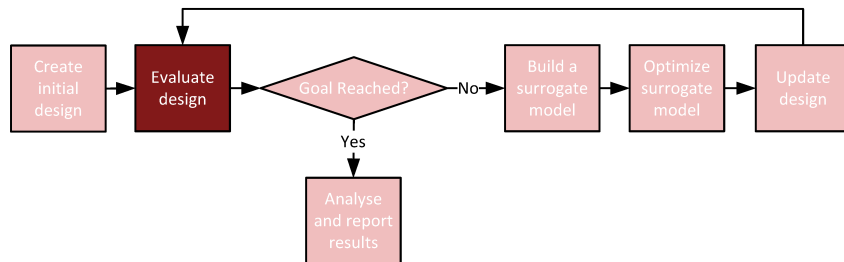
Beispiel



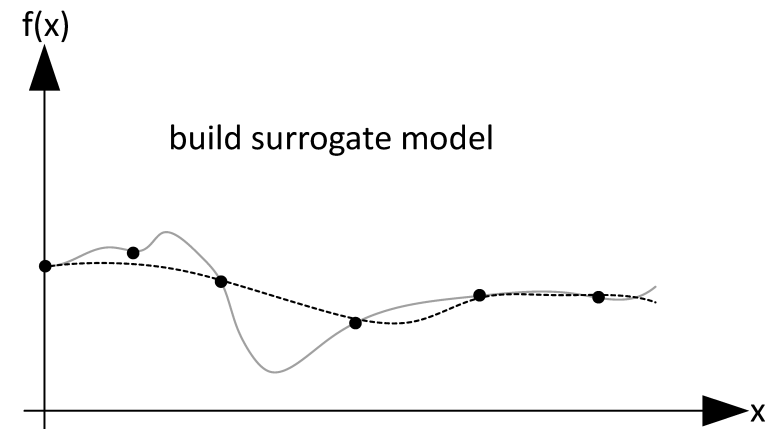
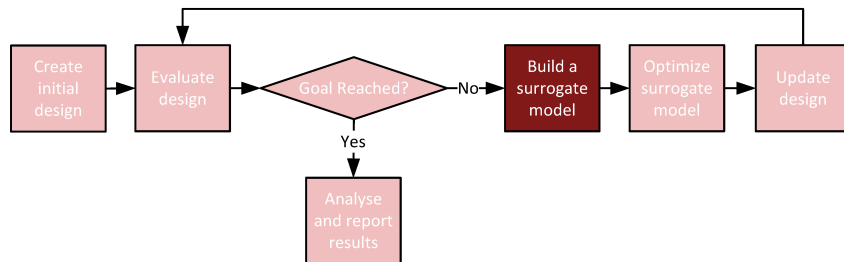
Beispiel



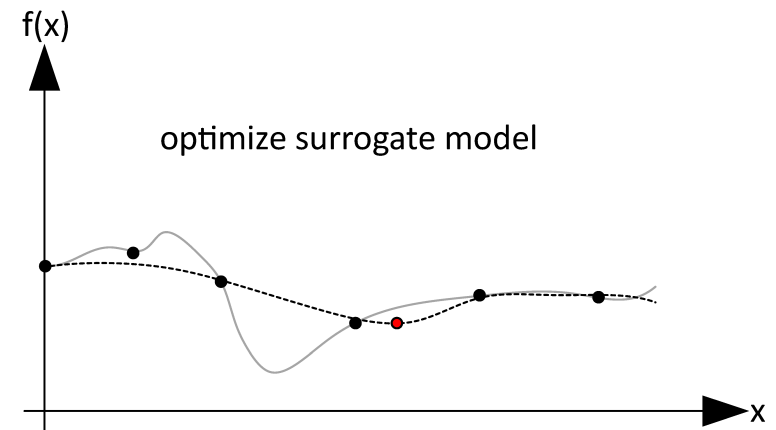
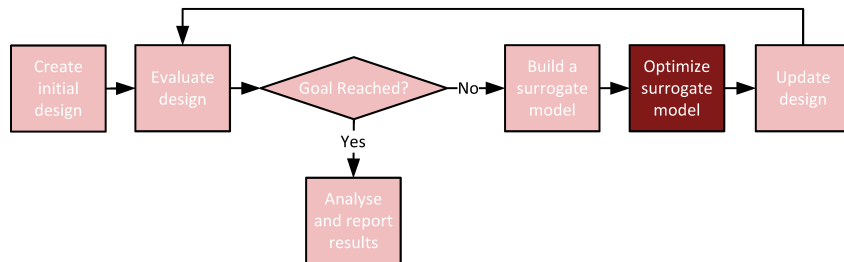
Beispiel



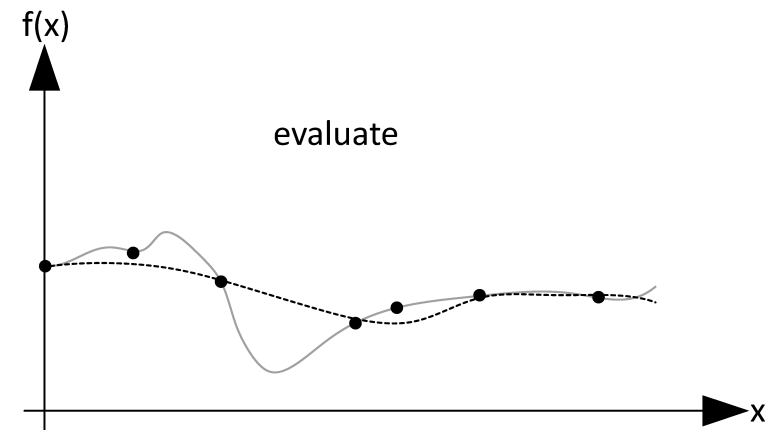
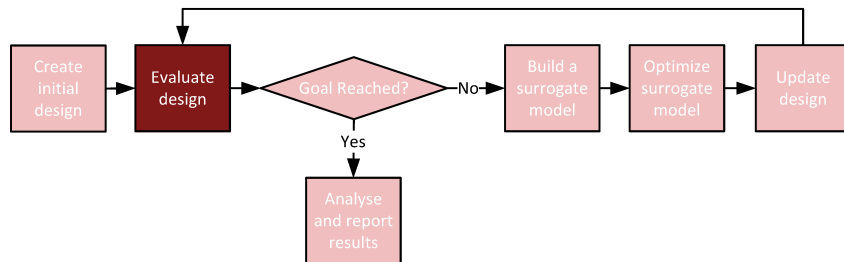
Beispiel



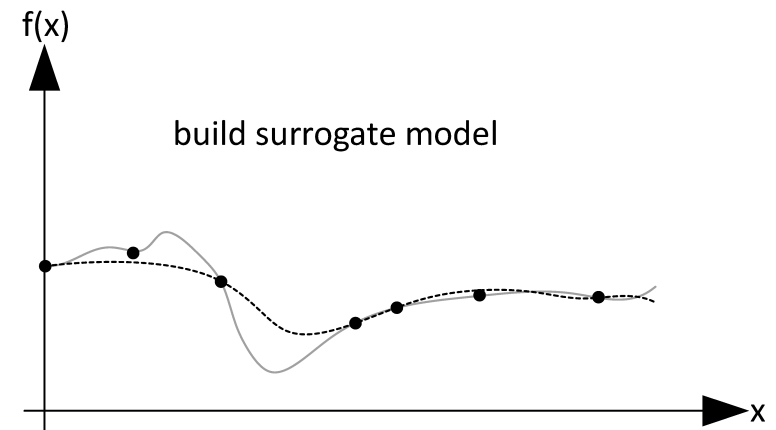
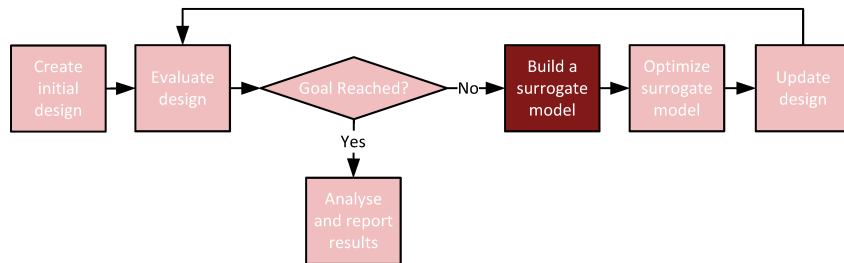
Beispiel



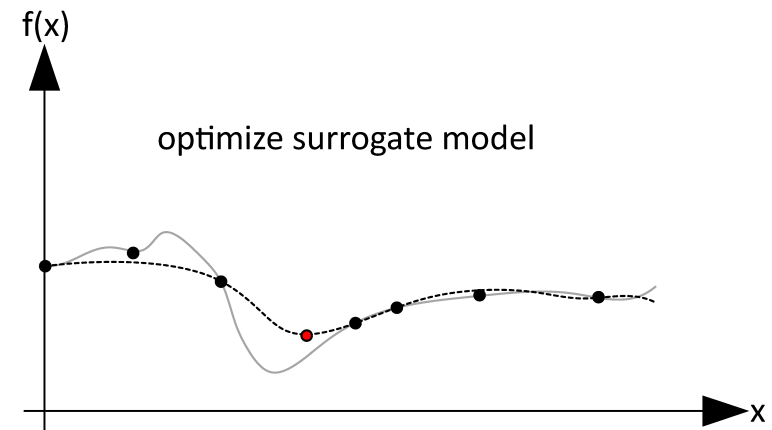
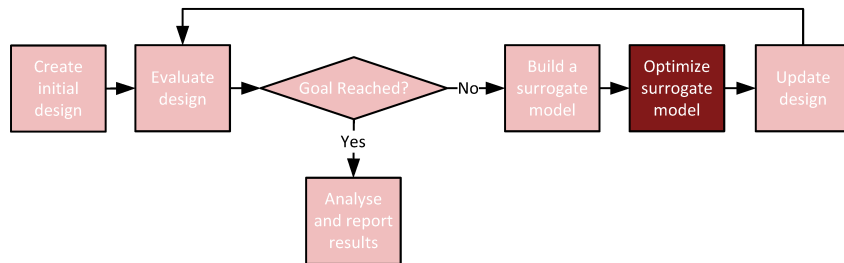
Beispiel



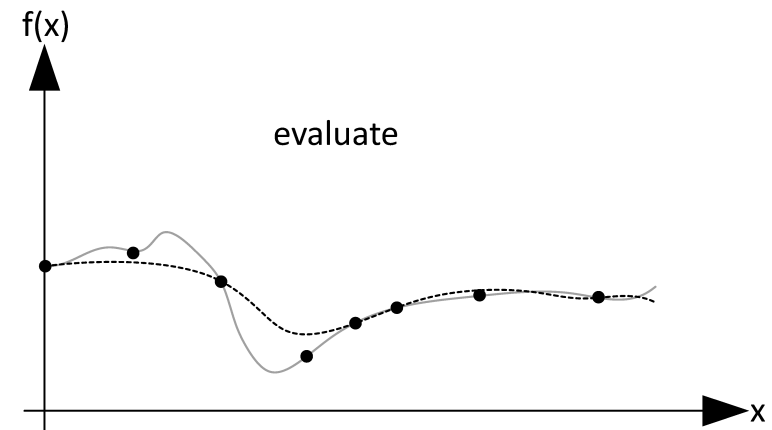
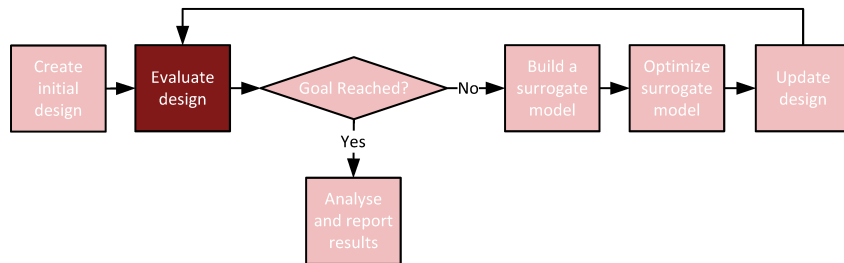
Beispiel



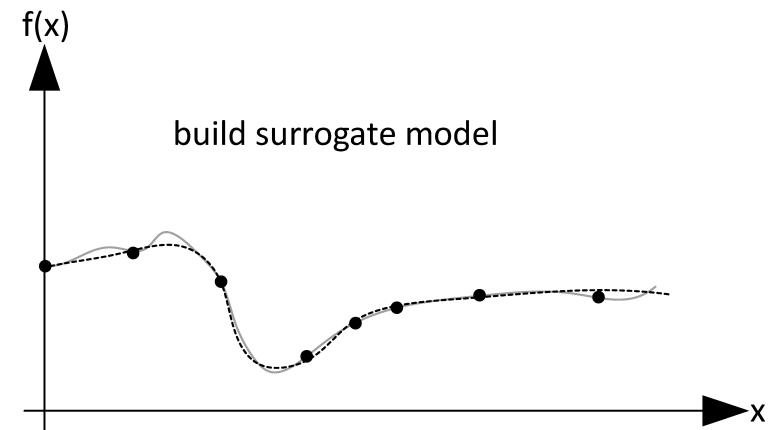
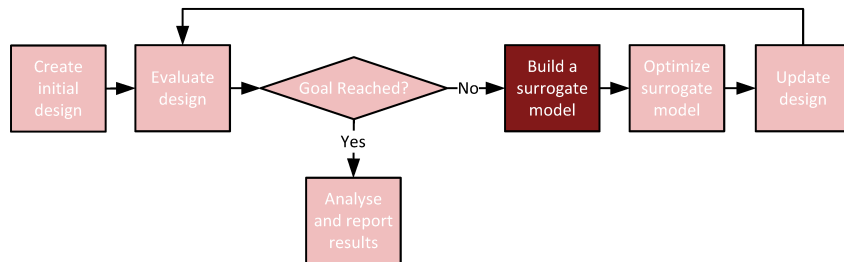
Beispiel



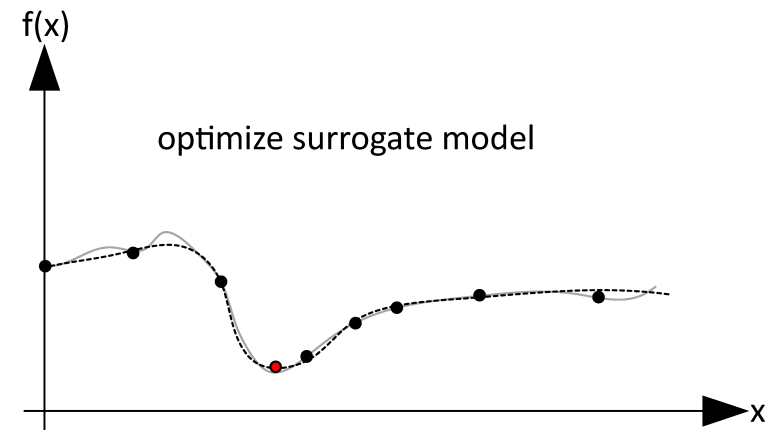
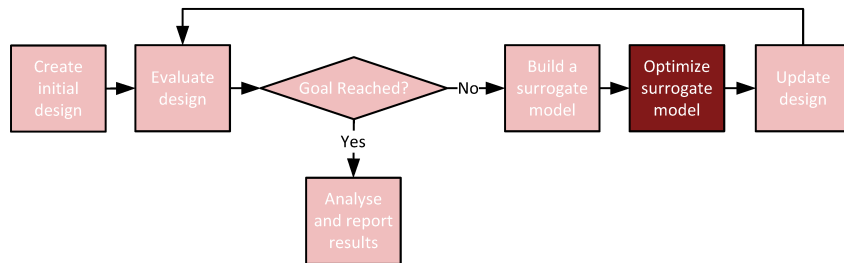
Beispiel



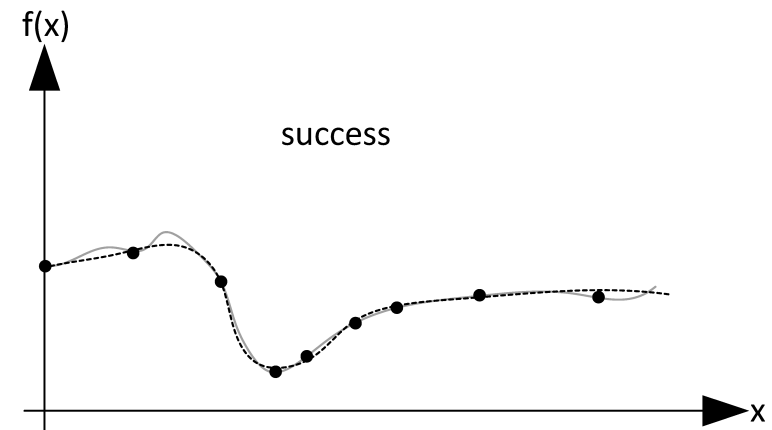
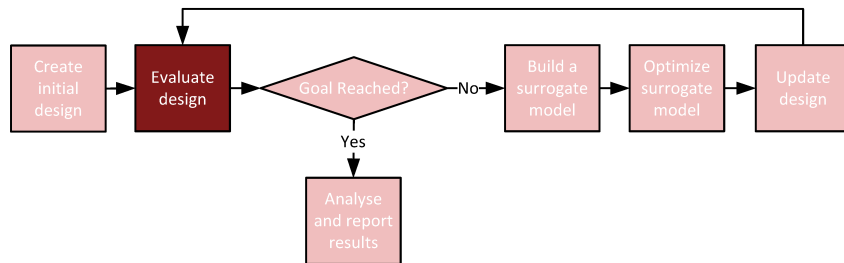
Beispiel



Beispiel



Beispiel



Überblick

Eigenschaften von Algorithmen/Methoden

	Methode	Global	Gradientenbasiert	Nebenbedingungen	Ziele	Linear	Ansatz	Genauigkeit	Deterministisch	Variablen
1	Stationäre Punkte / Hesse	✓	✓	ohne	1	✗	analytisch	exakt	✓	$\mathbb{R}^n$
2	Gradient Descent	✗	✓	ohne	1	✗	numerisch	?	✓	$\mathbb{R}^n$
3	Simplex	✗	✓	alle	1	✓	numerisch	exakt	✓	$\mathbb{R}^n$ & $\mathbb{Z}^n$
4	QO	✗	✓	alle	1	Nebenbed.	numerisch	?	✓	$\mathbb{R}^n$
5	KKT	✓	✓	alle	1		analytisch	exakt	✓	$\mathbb{R}^n$
6	SQO	✗	✓	alle	1		numerisch	?	✓	$\mathbb{R}^n$
7	Nelder-Mead	✗	✗	ohne	1		numerisch	approx.	✓	$\mathbb{R}^n$
8	DIRECT	✓	✗	Grenzen	1	✗	numerisch	approx.	✓	$\mathbb{R}^n$
9	Evolutionäre Algorithmen	✓ (?)	✗ (?)	/	/	✗	numerisch	approx.	✗	/
10	(1+1)-ES	✗	✗	ohne	1	✗	numerisch	approx.	✗	$\mathbb{R}^n$
11	CMA-ES	✓ (?)	✗	ohne	1	✗	numerisch	approx.	✗	$\mathbb{R}^n$
14	SMBO	✓ (?)	✗ (?)	/	/	✗	numerisch	approx.	/	/